

COMPENDIO TÁCTICO: DIVIDE Y VENCERÁS

Partición, Recurrencias y el Teorema Maestro (Edición Completa)

1. El Eje Teórico: La Ecuación de Recurrencia

Todo algoritmo de Divide y Vencerás (DyV) se resume en una fórmula matemática que describe cuánto tiempo tarda en ejecutarse. Si dominas esta fórmula, dominas el tema.

El Teorema Maestro (El Oráculo de la Complejidad)

Todo algoritmo DyV tiene una ecuación de tiempo de la forma:

$$T(n) = a \cdot T(n/b) + \mathcal{O}(n^d)$$

- **a: El Ejército** (Número de subproblemas en los que divides, ej. hago 2 llamadas recursivas $\rightarrow a = 2$).
- **b: El Hachazo** (Factor de reducción del tamaño. Si parto por la mitad $\rightarrow b = 2$).
- **d: El Pegamento** (El coste polinómico de *dividir* y *combinar*. Si un bucle recorre el array para fusionar $\rightarrow d = 1$. Si es un simple 'if' o suma $\rightarrow d = 0$).

La Resolución Directa (Aprender de Memoria):

- **Caso 1 (Gana el Ejército):** Si $a > b^d \implies \mathcal{O}(n^{\log_b a})$ (Ej: Multiplicación normal $\mathcal{O}(n^2)$, Strassen $\mathcal{O}(n^{\log_2 7})$).
- **Caso 2 (Empate Técnico):** Si $a = b^d \implies \mathcal{O}(n^d \log n)$ (Ej: Mergesort $a = 2, b = 2, d = 1 \implies \mathcal{O}(n \log n)$).
- **Caso 3 (Gana el Pegamento):** Si $a < b^d \implies \mathcal{O}(n^d)$ (El coste de juntar las piezas domina el algoritmo).

2. Metodología de Resolución: Los 3 Pasos Innegociables

Al igual que Backtracking tenía 5 pasos, DyV tiene 3 fases puras y duras. En un examen de diseño, debes justificar cada una:

■ PASO 1: DIVIDIR (El Corte)

- *La Lógica:* ¿Cómo parto el problema original en piezas más pequeñas?
- *Ejemplo:* Calculo el índice central $m = (inf + sup)/2$ para partir el vector en dos mitades exactas”.

■ PASO 2: VENCER (El Caso Base y la Llamada)

- *La Lógica:* ¿Cuándo el problema es tan pequeño que la respuesta es obvia sin tener que calcular nada? ¿Y si no es obvio, a quién llamo?
- *Ejemplo:* Caso base: si $inf == sup$, devuelvo el único elemento. Recursividad: llamo a la función para la mitad izquierda y para la mitad derecha”.

■ PASO 3: COMBINAR (El Pegamento)

- *La Lógica:* Ya tengo las soluciones mágicas de mis mitades. ¿Cómo las mezclo para obtener la respuesta total?
- *Ejemplo:* Comparo la solución de la izquierda con la de la derecha y devuelvo el máximo” (Coste $\mathcal{O}(1)$) o ”Hago un bucle fusionando dos arrays ordenados” (Coste $\mathcal{O}(n)$).

3. El Motor Único: El Arte de Rellenar los Cartuchos

Cualquier problema de DyV es una instancia del esquema general: resolver si es pequeño o descomponer y combinar si no lo es[cite: 30, 31, 38].

```

TipoRetorno dyv(Estructura E, int inf, int sup, variables_extra) {
    // CARTUCHO 1: Caso Base (El "Freno") -> Resolución directa [cite: 31]
    if ( esPequeño(inf, sup) ) return resolverDirectamente(E, inf, sup);

    // CARTUCHO 2: Dividir (El "Hachazo") -> Descomponer en sub-problemas [cite: 32]
    int medio = calcularPuntoCorte(inf, sup); // Ej: (inf + sup) / 2 [cite: 833]

    // CARTUCHO 3: Vencer (El "Ejército") -> Llamadas recursivas [cite: 34]
    // PRESET ALERTA: ¿Llamo a una rama (reducción) o a varias (división)?
    sol_1 = dyv(E, inf, medio, ...);
    sol_2 = dyv(E, medio + 1, sup, ...);

    // CARTUCHO 4: Combinar (El "Pegamento") -> Fusionar resultados [cite: 38]
    return combinar(sol_1, sol_2, ...);
}

```

3.1. Lenguaje Natural: Disparadores de Patrón

Para no atascarte en el examen, busca estas "señales".^{en} el enunciado que te dicen qué cartucho insertar:

- **Preset A: .^{EI} Francotirador** (Búsqueda Dicotómica / Reducción)
 - *Disparador*: "Busca un elemento...", Coste $O(\log n)$, "Vector ordenado" [cite: 801, 1104, 1107].
 - **Cartucho 3: Una sola llamada**. El `if` decide qué mitad descartar [cite: 826, 1104].
 - **Cartucho 4**: Vacío (`return` directo del subordinado) [cite: 826].
- **Preset B: .^{EI} General** (Exploración / Fusión)
 - *Disparador*: ".Ordena...", "Mayoría...", "Combinar resultados de mitades", Coste $O(n \log n)$ [cite: 840, 920, 1129].
 - **Cartucho 3: Doble llamada** obligatoria (izquierda y derecha) [cite: 861, 863, 946, 947].
 - **Cartucho 4**: El núcleo del código. Bucle `while/for` de fusión lineal $O(n)$ [cite: 864, 879, 919].
- **Preset C: .^{EI} Mago** (Optimización Aritmética)
 - *Disparador*: "Mejorar el coste clásico $O(n^2)$ ", "Multiplicación marciana", "Matriz de rastreo" [cite: 235, 541, 1174].
 - **Cartucho 3**: Múltiples llamadas estratégicas ($a = 3$ para Karatsuba, $a = 7$ para Strassen) [cite: 324, 364, 552, 575].
 - **Cartucho 4**: Sumas, restas y desplazamientos (*shifts*) algebraicos [cite: 354, 560, 1176].

4. Despliegue de los Cartuchos: Ejercicios del Boletín

A. El Punto Fijo $a[i] == i$ (Boletín Ej. 1)

- **Patrón**: Búsqueda Dicotómica (Variante). Objetivo: $\mathcal{O}(\log n)$.
- **Cartucho 1 (Caso Base)**: Si $inf > sup$, `return -1` (No existe). Si $inf == sup$, comprobamos si $a[inf] == inf$.
- **Cartuchos 3 y 4 (La Magia)**: Como el vector está ordenado estrictamente ($<$), si en la posición `medio` resulta que $a[medio] > medio$, sabemos seguro que hacia la derecha los números crecerán más rápido que los índices. Luego el punto fijo **tiene que estar a la izquierda**.
- **Código interno**: `if (a[mid] == mid) return mid; else if (a[mid] > mid) return dyv(inf, mid-1); else return dyv(mid+1, sup);`

B. Contar Unos en Vector Binario Ordenado (Boletín Ej. 3)

- **Patrón**: Búsqueda Dicotómica. Objetivo: $\mathcal{O}(\log n)$.
- **Lógica**: Como es un vector de 0s seguidos de 1s, contar los unos es equivalente a encontrar la posición del **primer 1** de todo el array. Si el primer 1 está en la posición k , la cantidad de unos es $n - k + 1$.
- **Código interno**: Si $v[mid] == 0$, los unos están seguro a la derecha `dyv(mid+1, sup)`. Si $v[mid] == 1$, cuidado, podría ser el primero o no. Miramos si $v[mid - 1] == 0$. Si lo es, ¡bingo! Es el primero. Si no, seguimos buscando hacia la izquierda `dyv(inf, mid-1)`.

C. Encontrar la Rotación k (Boletín Ej. 4)

- **Patrón:** Búsqueda Dicotómica. Objetivo: $\mathcal{O}(\log n)$.
- **Lógica:** El índice k de rotación es exactamente el único punto del array donde se produce una caída: $v[i] > v[i+1]$. Todo lo demás es ascendente.
- **Código interno:** Si $v[mid] > v[mid+1]$, hemos encontrado la caída, **return mid**. Si la mitad actual es ascendente respecto al origen ($v[mid] > v[inf]$), significa que el tramo raro (la caída) está a la derecha. Si es menor, está a la izquierda.

D. El Candidato Ganador (Boletín Ej. 5)

- **Patrón:** Fusión / Exploración Total con $a = 2, b = 2, d = 1 \implies \mathcal{O}(n \log n)$.
- **Cartucho 1 (Base):** Si el rango es de tamaño 1 ($inf == sup$), ese único voto es el ganador de esa mitad.
- **Cartucho 3 (Vencer):** Partimos el censo en dos. Llamamos recursivamente y obtenemos un **ganador_izq** y un **ganador_der**.
- **Cartucho 4 (Combinar - El Pegamento):** Si ambos lados dicen el mismo candidato, ese gana. Si dicen candidatos distintos, ¿quién es el ganador global? Recorremos todo el rango $[inf, sup]$ actual llamando a **esCoincidente** para contar cuántos votos tiene **ganador_izq** y cuántos tiene **ganador_der**. El que sume más de la mitad, lo subimos para arriba.

E. Aritmética Extraterrestre (Boletín Ej. 9)

- **Patrón:** Reducción de Coste Matemático (Estilo Karatsuba).
- **Ecuación Maestra:** El enunciado nos pide $\mathcal{O}(n^{\log_2 3})$. Esto nos chiva el Teorema Maestro: necesitamos obligatoriamente $a = 3$ (3 llamadas recursivas), $b = 2$ (partir los números por la mitad) y coste lineal de sumas $d = 1$.
- **El Truco de Combinación (Cartucho 4):** El operador es $\otimes(X, Y) = X^2 + XY + Y^2$.
- Si partimos $X = x_1 \cdot 10^m + x_2$ y $Y = y_1 \cdot 10^m + y_2$ (donde $m = n/2$).
- Para no calcular cuatro trozos como si fuera fuerza bruta, calculamos solo 3 recursiones:
 1. $R_1 = \otimes(x_1, y_1)$
 2. $R_2 = \otimes(x_2, y_2)$
 3. $R_3 = \otimes(x_1 - x_2, y_1 - y_2)$
- Usando álgebra de combinación, ensamblamos el número gigante utilizando sumas, restas y *shifts* (añadir ceros al final). ¡Es pura manipulación algebraica de Karatsuba!

8. Guía de Asociación Rápida: De la pregunta a la respuesta

Para resolver cualquier ejercicio del boletín, aplica esta tabla de decisión mental:

Si el enunciado dice...	El Patrón es...	Tu estrategia es...
Encuentre un índice i tal que $a[i] = i$ en $\mathcal{O}(\log n)$ [cite: 1104]	Francotirador	Mirar mid . Si $a[mid] > mid$, el índice i no puede estar a la derecha.
"Dos elementos cuya suma sea X en $\mathcal{O}(n \log n)$ " [cite: 1106]	General (Fusión)	Ordenar primero con Mergesort y luego usar dos punteros (uno al inicio y otro al final).
"Determine si la matriz es de rastreo" [cite: 1157]	Francotirador N-ario	Dividir en 4 submatrices. Comprobar si los bordes de $a_1..b_4$ cumplen la cadena de desigualdades [cite: 1140].
"Mediana de dos vectores en $\mathcal{O}(\log n)$ " [cite: 1139]	Francotirador Dual	Comparar medianas de X e Y . Descartar la mitad inferior de uno y la superior del otro.

Justificación de Coste para el Examen

Cuando diseñes el algoritmo, justifica siempre el coste usando el esquema de la pág. 11[cite: 182]:

- El coste es $O(\log n)$ porque dividimos el problema en $a = 1$ subproblema de tamaño $n/2$ con un coste de combinación constante $O(1)$ [cite: 186, 801].
- El coste es $O(n \log n)$ porque resolvemos $a = 2$ subproblemas de tamaño $n/2$ y el pegamento requiere un recorrido lineal $O(n)$ [cite: 186, 920].

9. Justificación del Coeficiente 'a' (Optimización)

Es fundamental explicar en el examen por qué el número de llamadas recursivas (a) disminuye en los algoritmos optimizados respecto a la fuerza bruta.

9.1. Multiplicación de Enteros (Karatsuba)

- **Intento Clásico** ($a = 4$): El primer intento DyV requiere 4 llamadas recursivas: $mult(x_1, y_1)$, $mult(x_1, y_2)$, $mult(x_2, y_1)$ y $mult(x_2, y_2)$ [cite: 274, 297, 298, 299]. Esto resulta en una complejidad cuadrática $O(n^2)$ [cite: 316].
- **Algoritmo multV2** ($a = 3$): Karatsuba utiliza variables auxiliares (p, q, r) para reducir el cálculo a solo 3 llamadas recursivas: $multV2(x_1 + x_2, y_1 + y_2)$, $multV2(x_1, y_1)$ y $multV2(x_2, y_2)$ [cite: 324, 331, 352, 353].
- **Ecuación de Recurrencia**: Al ser el coeficiente de la llamada recursiva un 3 ($T(n) \leq 3T(n/2) + k_2n$), el parámetro a es igual a 3[cite: 363, 364, 366].

9.2. Multiplicación de Matrices (Strassen)

- **DyV Estándar** ($a = 8$): Dividir matrices de $n \times n$ en 4 submatrices de $n/2 \times n/2$ requiere calcular 8 productos para obtener los cuadrantes C_{ij} [cite: 517, 520, 521]. Esto genera la recurrencia $T(n) = 8T(n/2) + k_2n^2$, manteniendo el coste clásico $O(n^3)$ [cite: 538, 541].
- **Idea de Strassen** ($a = 7$): Se reduce el número de multiplicaciones a costa de aumentar sumas y restas, calculando 7 matrices auxiliares (P, Q, R, S, T, U, V) que solo requieren 7 productos recursivos[cite: 550, 551, 552].
- **Ecuación de Recurrencia**: La fórmula definitiva $T(n) = 7T(n/2) + k_2n^2$ confirma que el coeficiente de recursión es 7, por lo tanto $a = 7$ [cite: 575, 579].

Resumen para Defensa

- **Karatsuba**: $a = 3$ porque se calculan solo tres productos de números de $n/2$ cifras[cite: 366].
- **Strassen**: $a = 7$ porque se calculan solo siete productos de submatrices de $n/2 \times n/2$ [cite: 579].